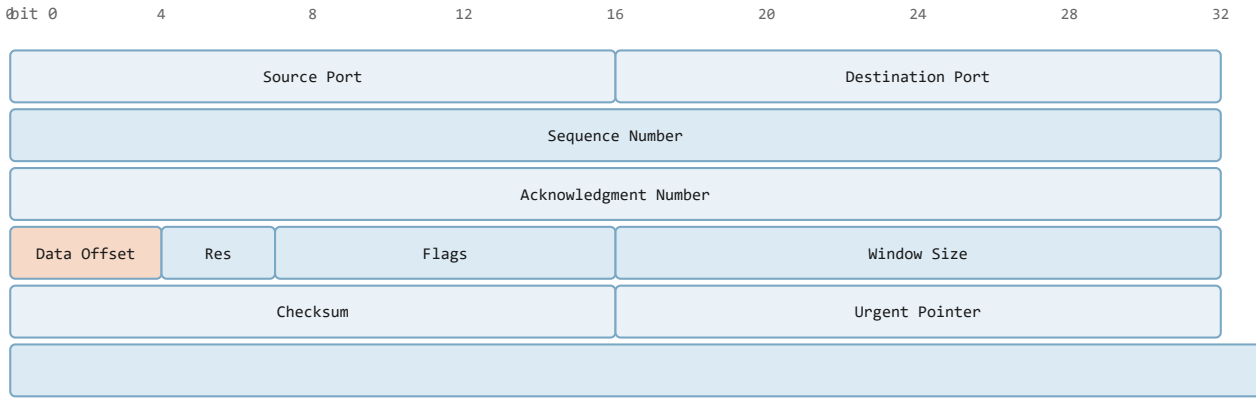
Clase 17 — TCP: la cabecera campo por campo

Bloque: C (Transporte) · **Semana:** 6 · **Duración estimada:** 110 min **Correlativa:** Clase 16 (mux/demux y UDP) · **Sigue con:** Clase 18 (TCP: establecer y cerrar la conexión) **Material de referencia:** Tanenbaum cap. 6 (§6.3, §6.5) · RFC 9293, 2018, 7323, 1122

🎯 Al terminar esta clase vas a poder

1. Decir qué servicios compra TCP con sus **20 bytes** de cabecera, frente a los 8 de UDP.
2. Escribir el **formato de la cabecera** y decodificar un segmento hexadecimal byte a byte.
3. Calcular **seq y ack de bytes**, y explicar por qué el ACK es **acumulativo** y cuenta bytes.
4. Explicar qué significa realmente **PSH** y por qué **no** garantiza un mensaje por mensaje.
5. Describir las **opciones** (MSS, SACK permitted, Timestamps, Window Scale) con su RFC.
6. Calcular y verificar el **Internet checksum** de un segmento, y decir qué pasa si falla.

Cabecera TCP, campo por campo



Cabecera minima: 5 filas de 32 bits = 20 bytes. Con opciones puede llegar a 60 bytes.

> **Prerequisito del examen:** `2011-02 P3` y `2007-03 P4` (10 pts cada uno, campos de TCP y UDP) · `2001-12 Problema 2` (el PUSH no garantiza un mensaje por vez).

PARTE 1 — TEORÍA

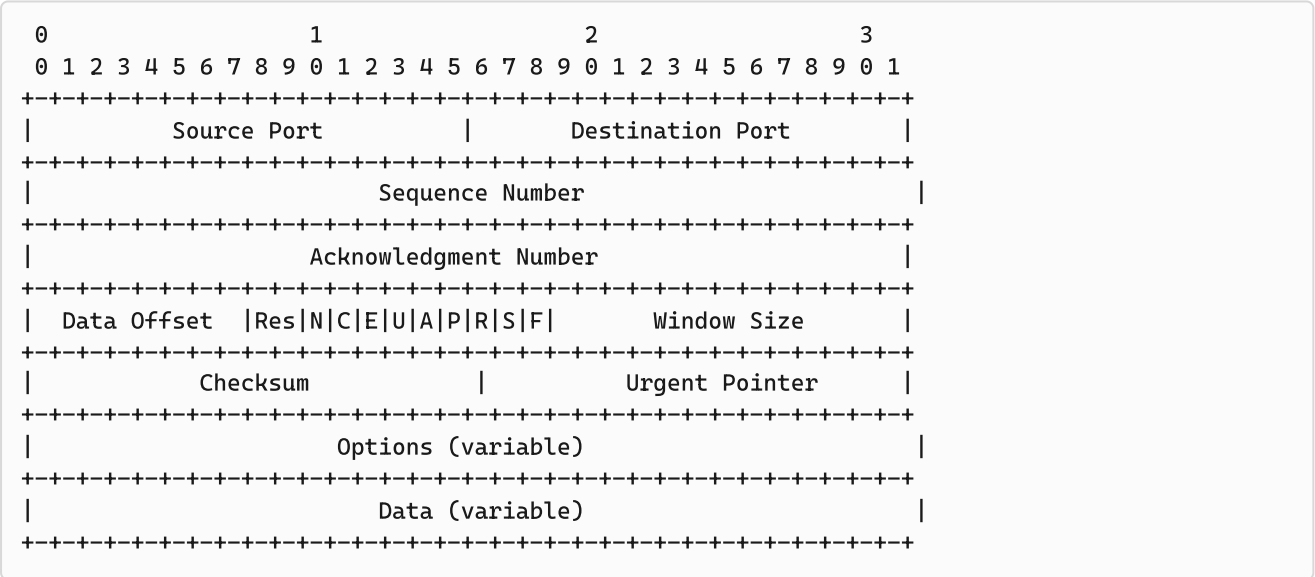
1.1 Por qué 20 bytes y no 8

UDP tiene 8 bytes y no garantiza nada. TCP tiene 20 y compra con ellos:

Bytes	Campo	Qué le compra al protocolo
4	Puertos (2 + 2)	saber a qué proceso es el flujo
8	Sequence + Acknowledgment	**orden, detección de pérdida y retransmisión**
2	Window	**control de flujo** (cuánto puede mandar sin desbordar al receptor)
2	Checksum	**detección de errores** de punta a punta, con pseudo-cabecera
2	Data Offset + Reserved + Flags	**ciclo de vida de la conexión** (SYN, ACK, FIN, RST, PSH)
2	Urgent Pointer	datos urgentes (URG), que en la práctica no se usa

Sobrevivir sin conexión, en cambio, es gratis: TCP no lo necesita porque **el número de secuencia no identifica una conexión, identifica un byte**. No tiene que preguntarle "¿de quién es este paquete?", porque el socket ya lo sabe. Esa es la diferencia conceptual más importante entre los dos.

1.2 El formato de la cabecera



5 palabras de 32 bits = 20 bytes sin opciones; como las opciones son de largo variable, hace falta el **Data Offset** para saber dónde empiezan los datos.

El byte 12 tiene los 4 bits del Data Offset, 3 reservados y el bit NS (*No Significant*, de `RFC 7323`); **el byte 13**, los 8 flags:

El byte 13 son los flags, y cada uno es **un bit**:

Bit	Valor	Flag	Significado
7	`0x80`	CWR	obsoleto (era de *congestion window reduced*)
6	`0x40`	ECE	obsoleto (era de *ECN-Echo*)
5	`0x20`	**URG**	el puntero urgente es válido
4	`0x10`	**ACK**	el campo Acknowledgment es válido
3	`0x08`	**PSH**	no esperes a llenar el buffer para mandar
2	`0x04`	**RST**	resetear la conexión / error
1	`0x02`	**SYN**	sincronizar números de secuencia (abrir)
0	`0x01`	**FIN**	no tengo más datos (cerrar)

CWR y ECE están **reservados** desde que ECN se negocia en el SYN (`RFC 3168`), y **si ACK = 0 el campo Acknowledgment se ignora**: por eso el SYN puede viajar con `ack = 0`.

1.3 Un segmento real, byte a byte

Segmento del cliente `203.0.113.9:49200` al servidor `198.51.100.7:80`, con 4 bytes de datos ("HOLA"), sin opciones:

```
Byte   0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19
     C0 30 00 50 00 00 04 4C 00 00 04 60 50 18 20 00 CB C6 00 00
      └────┘ └────┘ └──── seq ────┘ └──── ack ────┘ └┘ └─────┘ └──ck──┘ └urg┘
      sport  dport = 1100     = 1120    offset flags  window
     C0 30 = 49200   00 50 = 80   00 00 04 4C = 1100   00 00 04 60 = 1120
     50 = 0101 0000  →  Data Offset 5 (= 20 bytes), reservado 0, NS 0
     18 = 0001 1000  →  ACK (0x10) | PSH (0x08)      20 00 = ventana 8192
     CB C6 = checksum (calculado en 2.2)  00 00 = urgente nulo
     datos: 48 4F 4C 41 = "HOLA"
```

Las conversiones, con la cuenta a la vista:

```
  49200 = 0xC030   (0xC000 = 49152  +  0x30 = 48)     80 = 0x0050
   1100 = 0x0000044C   (4·256 = 1024  +  0x4C = 76)   1120 = 0x00000460 (4·256 + 96)
     20 = 0x14  →  Data Offset = 0x14 / 4 = 5  →  5 × 4 = 20 bytes de cabecera
   8192 = 0x2000   (2 × 4096)       PSH|ACK = 0x08 | 0x10 = 0x18
```

El puntero urgente: si URG = 1, la secuencia urgente es `seq + puntero`. En 2026 **nadie lo usa**: ni aplicaciones ni bibliotecas lo activan.

1.4 Sequence Number y Acknowledgment Number: la aritmética

Los dos campos son de **32 bits** y el ACKing es **acumulativo** (*cumulative ACK*), y por eso el número identifica **bytes**, no segmentos. Esta es la clase de error que más puntos cuesta.

Caso base: el cliente manda segmentos de **20 bytes** desde `seq = 1100`.

```
bytes enviados:  1100 → 1119    1120 → 1139    1140 → 1159    1160 → 1179
seq del seg.:        1100           1120           1140           1160
ACK que devuelve:    1120           1140           1160           1180
                     ↑              ↑              ↑              ↑
                     el receptor dice "espero el 1120": ya tengo 1100–1119
```

El ACK es siempre "el próximo byte que espero", no "el último que recibí". Por eso el primer segmento produce `1120 = 1100 + 20`.

Ahora se pierde el segundo segmento:

```
seg1 (1100–1119) llega → receptor responde  ACK 1120   → el emisor avanza a 1120
seg2 (1120–1139) SE PIERDE  → el receptor sigue esperando 1120
seg3 (1140–1159) llega → ¿lo acepta? NO: llega fuera de orden.
                         → el receptor responde ACK 1120  (¡DUPLICADO!)
retransmisión de seg2 → ACK 1140
seg4 (1160–1179) llega → ACK 1180
```

Los **ACKs duplicados** (`1120`, `1120`, `1120`) evidencian que falta un segmento y disparan el *fast retransmit* de la clase 21: con 3 se retransmite sin esperar el RTO.

Consecuencia: un solo ACK puede confirmar **varios segmentos a la vez**, y uno puede quedar confirmado **parcialmente**: la unidad es **siempre el byte**.

1.5 El flag PSH: lo que dice y lo que no dice

Acá está el malentendido clásico, y es lo que el examen 2001-12 pregunta explícitamente.

En el receptor, PSH no obliga a nada: es una *pista* (*hint*) para el stack local. La decisión de llamar a `read()` y devolver uno o varios mensajes es **de la aplicación y del buffer del socket**, y entre el cable y el `read()` hay **tres capas de buffering**: socket, reordenamiento y transporte. En el emisor, además, está el **WAL**.

Las tres razones por las que PUSH no garantiza nada:

1. **Segmentación.** Si el mensaje es más grande que el MSS, TCP lo parte en varios segmentos: el PSH no lo impide.
2. **WAL (*write-ahead log*) en el emisor.** El PSH queda marcado en un buffer y puede no salir si la ventana está llena o si Nagle está activo: se "pierde" hasta que el segmento sale.
3. **Buffering en el receptor.** Con dos mensajes en el buffer del socket, **un solo `read()` puede devolver los dos**; y con reordenamiento, el final del 2 junto con el principio del 1.

En la práctica **todas las implementaciones lo ponen siempre y casi ninguna receptora lo trata de forma especial**: hoy es decorativo.

1.6 Las opciones

Cada opción tiene la forma `Kind (1 byte) + Length (1 byte) + datos`, y el **Length incluye los dos primeros bytes**. Si el total no es múltiplo de 4 se completa con **NOP (Kind 1)**; el fin se marca con **EOL**

(Kind 0).

Opción	Kind	Long.	Qué negocia	RFC
EOL	0	—	fin de las opciones	RFC 9293
NOP	1	1	alineación	RFC 9293
MSS (Maximum Segment Size)	2	4	máximo de bytes de datos **por segmento**	RFC 9293 (§3.7.1)
Window Scale	3	3	multiplica la ventana ×2^shift	`RFC 7323`
SACK Permitted	4	2	"sé usar ACK selectivo"	`RFC 2018`
Timestamps	8	10	TSval y TSecr para medir RTT y proteger PAWS	`RFC 7323`

En un SYN se ven así:

```
02 04 05 B4   →  MSS = 0x05B4 = 1460 bytes de datos por segmento
04 02         →  SACK permitted (2 bytes, sin datos)
08 0A 00 00 1E D4 00 00 00 00
              →  Timestamps: TSval = 0x1ED4 = 7884, TSecr = 0
Total = 4 + 2 + 10 = 16 bytes  →  Data Offset = 5 + 16/4 = 9  →  36 bytes
```

Las tres que importan:

- **MSS**: máximo de **datos**, no de segmento. Con IP y Ethernet (MTU 1500) sale 1500 − 20 − 20 = **1460**. Se anuncia **solo en el SYN** y gana el **menor**: es lo que evita la fragmentación de IP.
- **SACK permitted** (`RFC 2018` , *Standards Track*): es solo el **permiso**. Si los dos lo mandan, después un segmento puede llevar campos SACK con los bloques que faltan. Sigue siendo comportamiento estándar y es lo que usan Linux y Windows.
- **Timestamps** (`RFC 7323` , que **obsolesce a la RFC 1323**): 10 bytes con **TSval** (el reloj del emisor) y **TSecr** (su eco). Sirven para medir el RTT exacto y para **PAWS** (*Protect Against Wrapped Sequence numbers*), **obligatorio** si ambos extremos los activan.

1.7 El checksum de TCP: Internet checksum, no CRC

El pseudo-cabecera es el mismo de la clase 16, con el protocolo = **6**:

```
src (4)  dst (4)  zero (1)  protocolo = 6 (1)  longitud del segmento TCP (2)
```

La diferencia con UDP: la longitud es la del **segmento TCP completo**, cabecera + opciones + datos (en el ejemplo, 20 + 4 = 24 = `0x0018`).

Y el punto conceptual que se pregunta: **es un checksum, no un CRC.** Son algoritmos distintos:

	Internet checksum (TCP/UDP/IP)	CRC (Ethernet)
Qué es	**aditivo**: suma y complemento a uno	**polinómico**: división en campo de Galois
Errores de 1 bit	**siempre los detecta**	siempre
Errores de varios bits	solo los que no se compensan	sí, incluidos los *bursts*
Coste	sumar	tabla + divisiones

Por qué detecta todos los errores de un bit: cada bit aporta un `±2^k` distinto (k = 0..15), así que alterar un bit cambia la suma en un valor que **nunca da cero** en complemento a uno. Con dos o más bits, las contribuciones pueden **cancelarse**.

1.8 Qué pasa si el checksum falla

El segmento se descarta en silencio. No hay NAK, no hay ICMP, no hay aviso. ¿Por qué? (a) un NAK por cada datagrama corrupto genera una **tormenta de ACKs negativos**; (b) no arregla nada, porque un NAK perdido es tan inútil como el segmento perdido; y (c) el checksum existe para **no entregar datos malos**, no para informar.

Para el emisor, un segmento corrupto es indistinguible de uno perdido: se dispara el mismo mecanismo (RTO o 3 ACKs duplicados) y se retransmite. O sea: **el checksum convierte errores en pérdidas**, y para TCP eso es transparente.

1.9 ⚠️ Actualización a 2026

Sigue igual — de memoria: 20 bytes mínimos; los 10 campos; seq y ack de **32 bits y de bytes**; ACK **acumulativo**; ventana de **16 bits**; los 6 flags; Data Offset en palabras de 32 bits; Internet checksum con pseudo-cabecera; y ante un checksum roto, **descarte en silencio**.

Cambió o hay que corregirlo:

- `RFC 9293` **(2022, *Internet Standard*) sustituye a la** `RFC 793` **de 1981** (consolida 793 + RFC 879, 2873, 6093, 6429, 6528 y 6691). Citar "RFC 793" como especificación vigente en 2026 está desactualizado; `RFC 1122` sigue definiendo el comportamiento de los hosts.
- **Timestamps y Window Scale:** `RFC 7323` **(2014)**, que **obsolesce a la RFC 1323**, y **PAWS es obligatorio** si ambos activan timestamps: con 32 bits y enlaces rápidos, un paquete viejo podría aceptarse como nuevo.
- **SACK:** `RFC 2018` **(1996)**, *Standards Track*, **sigue siendo** el mecanismo estándar.
- **URG está obsoleto en la práctica**: ninguna aplicación ni biblioteca moderna lo usa. Y **PSH es decorativo**: casi todos los stacks lo mandan siempre. **QUIC elimina el problema de raíz**, porque los *streams* de HTTP/3 tienen **frontera de mensaje explícita**: un mensaje nunca se parte ni se junta con otro, sin depender de un flag.

Consecuencia para el examen: `2011-02 P3` y `2007-03 P4` **se siguen preguntando tal cual**: la respuesta esperada es la lista de campos con su significado. Para sumar, citá **RFC 9293** en lugar de RFC 793.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Data Offset y largo total de un segmento

Enunciado. Capturás un segmento TCP con estos primeros 20 bytes:

```
C0 30 00 50 00 00 04 4C 00 00 04 60 70 18 FA F0 12 34 00 00
02 04 05 B4 04 02 08 0A 00 00 1E D4 00 00 00 00
48 4F 4C 41 41 42 43 ...
```

a) ¿Cuánto vale el Data Offset? **b)** ¿Cuántas opciones hay y cuáles? **c)** ¿Cuánto mide el segmento completo si el datagrama IP mide 68 bytes? **d)** ¿Qué flags tiene?

a) El Data Offset. El byte 12 es `0x70 = 0111 0000` → Data Offset = **7** → **7 × 4 = 28 bytes** de cabecera (reservados `000`, NS `0`).

b) Las opciones. Desde el byte 20 hasta el 27 inclusive, o sea `28 − 20 = 8` bytes:

```
02 04 05 B4     →  Kind 2 (MSS), Length 4, valor 0x05B4 = 1460
04 02           →  Kind 4 (SACK permitted), Length 2
── y sobran 2 bytes sin EOL ni NOP: son padding, u opciones mal formadas.
```

Interpretación: el Data Offset 7 con solo 8 bytes de opciones visibles deja 8 bytes de hueco; en una captura real eso es padding. Lo que **no** puede pasar es que el Data Offset apunte más allá de los datos.

c) El largo total. El datagrama IP mide 68 bytes; le restamos la cabecera IPv4 (20): `68 − 20 = 48` bytes de segmento TCP, de los cuales 28 son cabecera ⇒ **20 bytes de datos**. *No hay contradicción con el MSS de 1460*: el MSS es un **máximo**, y se mide en **datos**, sin la cabecera.

d) Los flags. El byte 13 es `0x18`:

```
0x18 = 0001 1000  →  ACK (bit 4 = 0x10) | PSH (bit 3 = 0x08)
→  PSH + ACK, y no es SYN ni FIN ni RST
```

2.2 Ejercicio 2 — Calcular y verificar el Internet checksum de un segmento TCP

Datos. Segmento de `203.0.113.9` a `198.51.100.7`, protocolo 6, longitud **24** bytes: la cabecera de 20 del ejemplo de §1.3 (`seq = 1100`, `ack = 1120`, ventana 8192, flags `0x18`) y 4 de datos "HOLA".

a) Escribí las palabras a sumar. **b)** Sumá. **c)** ¿Cuál es el checksum? **d)** Verificá.

a) Las palabras (con el checksum en 0, que es como se calcula):

```
Pseudo-cabecera  CB00  7109  C633  6407  0006  0018    src, dst, proto 6, long. TCP 24
Cabecera TCP     C030  0050  0000  044C  0000  0460    ports, seq 1100, ack 1120
                 5018  2000  0000  0000               offset+flags, ventana, ck=0, urgente
Datos            484F  4C41                           "HO", "LA"
```

b) La suma con acarreo devuelto:

```
 CB00 + 7109 = 13C09  → 3C0A
 3C0A + C633 = 1023D  → 023E
 023E + 6407 = 06645  → 6645
 6645 + 0006 = 0664B  → 664B
 664B + 0018 = 06663  → 6663
 6663 + C030 = 12693  → 2694
 2694 + 0050 = 026E4  → 26E4
 26E4 + 0000 = 026E4  → 26E4
 26E4 + 044C = 02B30  → 2B30
 2B30 + 0000 = 02B30  → 2B30
 2B30 + 0460 = 02F90  → 2F90
 2F90 + 5018 = 07FA8  → 7FA8
 7FA8 + 2000 = 09FA8  → 9FA8
 9FA8 + 0000 = 09FA8  → 9FA8      ← el checksum, en 0
 9FA8 + 0000 = 09FA8  → 9FA8      ← puntero urgente
 9FA8 + 484F = 0E7F7  → E7F7
 E7F7 + 4C41 = 13438  → 3439
 ──────────────────────────────────────────
 suma final = 0x3439
```

c) El checksum es el complemento a uno de 16 bits:

```
0xFFFF − 0x3439 = 0xCBC6
```

d) Verificación. `3439 + CBC6 = FFFF`, sin acarreo, y el complemento de `FFFF` es `0x0000`: **el mismo criterio que en UDP, `0x0000` significa "segmento íntegro"**.

2.3 Ejercicio 3 — Si el checksum falla

Enunciado. Al segmento del ejercicio 2.2 le llega un **único bit alterado** en los datos: el byte `4F` de "HO" pasa a `50`. **a)** ¿Lo detecta el receptor? **b)** ¿Qué hace? **c)** ¿Qué le pasa al emisor?

a) La palabra `484F` pasa a `4850`, o sea **+1** en la suma:

```
suma original = 0x3439
suma nueva    = 0x3439 + 1 = 0x343A
checksum que calcularía el receptor = 0xFFFF − 0x343A = 0xCFB7
checksum que venía en el segmento   = 0xCBC6
0xCFB7 ≠ 0xCBC6  →  HAY ERROR
```

Interpretación: con **un solo bit** la diferencia es siempre ± `2^k`, que en complemento a uno nunca da cero: por eso se detectan todos los errores de un bit.

b) El receptor **descarta el segmento en silencio**: no ACK, no NAK, no ICMP. Los datos no llegan a la aplicación.

c) El emisor **no se entera de nada** hasta que expira el RTO, o hasta que recibe 3 ACKs duplicados. Y como un segmento corrupto es indistinguible de uno perdido, TCP lo **retransmite** igual. **El checksum convierte errores de bits en pérdidas, y el mecanismo de pérdida es transparente para la aplicación.** Ese es el diseño: el checksum protege la integridad, la retransmisión protege la entrega.

Y el caso inverso, para entender por qué no alcanza: si se alteran **dos bits de igual peso en palabras distintas** (por ejemplo, el bit 0 de `C030` hacia 1 y el bit 0 de `C633` hacia 0), las contribucciones son `+1` y `−1`, se cancelan, la suma da lo mismo y **el receptor acepta el segmento dañado**. Por eso TCP no confía en el checksum para detectar corrupción: confía en que la red no corrompe, y el checksum es la primera línea, no la última.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2011-02, Pregunta 3 (10 puntos) — Campos más importantes de TCP y UDP

> **Fuente:** Examen `eirc1102` / `sirc1102` — 3 de febrero de 2011.

Enunciado (texto del examen)

> Mencione y explique los campos más importantes de los segmentos TCP y UDP.

Solución oficial

La solución publicada **remite a una diapositiva del teórico** (`cap03_3_2009_2xcarilla.pdf`, slide 5): no hay texto de respuesta. Lo que sigue es **solución propia**.

Mi resolución (solución propia)

UDP — 4 campos, 8 bytes: puertos origen y destino (16 bits cada uno, que con la IP dan la 2-tupla del socket no conectado); **Length** (longitud **total** del datagrama, mínimo 8); **Checksum** (Internet checksum **con pseudo-cabecera**, opcional en IPv4 y obligatorio en IPv6; si falla se **descarta en silencio**); y los **datos** (0 a 65 507 bytes).

TCP — los 10 campos, 20 bytes:

1. **Source Port / Destination Port** (16 + 16): con las dos IP forman la **4-tupla** de la conexión.
2. **Sequence Number** (32): número del **primer byte**; cuenta **bytes**, no segmentos.
3. **Acknowledgment Number** (32): **próximo byte esperado**, **acumulativo**; solo vale si ACK está activo.
4. **Data Offset** (4 bits): largo de la cabecera en **palabras de 32 bits**, que marca dónde empiezan los datos.
5. **Reservado** (3 bits) y **NS** (1 bit), sin uso real.
6. **Flags** (8 bits): **URG, ACK, PSH, RST, SYN, FIN** (más CWR/ECE reservados).
7. **Window Size** (16): bytes que puede mandar el emisor desde el último confirmado → **control de flujo**; con *Window Scale* se multiplica por 2^shift.
8. **Checksum** (16): Internet checksum con **pseudo-cabecera de protocolo 6** y longitud del **segmento TCP**.
9. **Urgent Pointer** (16): desplazamiento del dato urgente desde `seq`, si URG está activo.
10. **Options**: MSS, SACK permitted, Window Scale, Timestamps, que se anuncian **en el SYN**.

Tope para 10 puntos: los 4 de UDP y 8 o 10 de TCP, cada uno con su tamaño en bits y una frase. La diferencia clave: **UDP identifica un socket; TCP identifica una conexión**, y por eso necesita números de

secuencia, ventana y flags de ciclo de vida.

3.2 Examen 2007-03, Pregunta 4 (10 puntos) — Campos más importantes de TCP y UDP

> **Fuente:** Examen `eirc0703` / `sirc0703` — 26 de febrero de 2007.

Enunciado (texto del examen)

> Mencione y explique los campos más importantes de los segmentos TCP y UDP.

Solución oficial (verbatim, resumida)

Enumera, en el orden de la diapositiva del teórico, **13 ítems de TCP** (puertos, seq/ack, largo del cabezal, URG, ACK, PSH, RST, SYN, FIN, ventana, checksum, opciones, datos) y **3 de UDP** (puertos, largo del cabezal, checksum).

Análisis

Bien: la lista es completa y sigue el orden del formato de la cabecera, que es como hay que responder.

Tres cosas que un corrector riguroso puede marcar:

1. **PSH.** La solución oficial dice que indica que "los datos deben ser enviados a la capa de aplicación inmediatamente luego de ser recibidos". Eso **describe el efecto observable** pero no el **mecanismo**, y por eso lleva a la confusión clásica: lo correcto es que PSH es un aviso del emisor al stack receptor de que **no espere a llenar el buffer**, y **no** obliga a la entrega inmediata a la aplicación ni garantiza que un mensaje llegue entero.
2. **"El puerto más la dirección IP forman un único punto de acceso."** Correcto para un socket UDP no conectado, pero incompleto: la clave real del kernel es la **5-tupla**, y la conexión TCP es la **4-tupla**.
3. **"Utilizar selective repeat en lugar de go back n."** Es una glosa de la opción **SACK**, pero SACK **no es** Go-Back-N ni Selective Repeat de capa 2: es **retransmisión selectiva de bytes sobre un protocolo que igual entrega en orden** (`RFC 2018`). Si lo decís así sumás; si lo decís como "selective repeat", restás.

3.3 Examen 2001-12, Problema 2 — Por qué PUSH no garantiza un mensaje por vez

> **Fuente:** Examen `eirc0112` — 27 de diciembre de 2001.

Enunciado (texto del examen)

> Es sabido que, al enviar mensajes mediante una conexión TCP, el mero uso del bit de PUSH no asegura que, en el destino, la capa de aplicación reciba un mensaje por vez. Se desea realizar una prueba de laboratorio que demuestre este concepto. Se pide construir un bridge que interconecte los equipos de origen y destino, y manipule el orden de transmisión de los paquetes y eventualmente la segmentación de los mismos (como podría ocurrir en una red real) de forma que en el destino se experimenten las siguientes situaciones: a) Arribo simultáneo de dos mensajes a la capa de aplicación de destino. b) Arribo simultáneo de un mensaje y de parte del siguiente a la capa de aplicación de destino.

Solución oficial (verbatim)

> a) Cambiar el orden de los mensajes. Se envían el 1 y el 2, ambos con push, pero se recibe primero el 2. Al recibirse el 1 se enviarán ambos a la capa de aplicación. b) Como en a) pero segmentando además los mensajes. Alcanza con que al equipo receptor llegue primero el comienzo del mensaje 2 y luego el mensaje 1 completo.

Mi resolución, explicada

a) Dos mensajes en un solo `read()`. El emisor manda `MSG1` y `MSG2`, ambos con PSH. El bridge **los reordena** y entrega primero el de `MSG2`, que queda esperando en el buffer del socket porque la aplicación todavía no hizo `read()`. Cuando llega `MSG1`, el stack lo **reencola por número de secuencia** y las dos cosas quedan **contiguas**: un solo `read()` devuelve los dos mensajes pegados. **Los dos segmentos tenían PSH y ninguno garantizó nada.**

b) Un mensaje entero y parte del siguiente. El bridge además **segmenta** `MSG2`, reordena, y hace llegar primero su **primer segmento** (digamos 100 bytes) y después el `MSG1` completo. El buffer queda `MSG1 | 100 bytes de MSG2`, todo contiguo: **un `read()` devuelve un mensaje entero y el comienzo del siguiente**.

Las tres razones de fondo (§1.5), que es la parte teórica que hay que escribir:

1. **Segmentación**: el PSH no impide que un mensaje se parta si es más grande que el MSS.
2. **WAL del emisor**: el PSH se marca en el buffer de envío y puede salir tarde si la ventana está llena o si Nagle está activo.
3. **Buffering del receptor**: el paso de bytes a la aplicación lo deciden el `read()` y el buffer del socket. El stack solo ve un **flujo de bytes**: **la frontera de mensaje no existe en TCP**, es un concepto de la capa de aplicación.

3.4 Examen 2001-12, Problema 1 — Un router que interrumpe una conexión TCP sin que se entere

> **Fuente:** Examen `eirc0112` — 27 de diciembre de 2001. *Se resume la idea; el enunciado completo y el pseudocódigo están en* `sirc0112`.

El problema. A tiene dos conexiones TCP hacia los puertos 9001 y 9002 de B y manda datos continuamente; B solo contesta ACKs. En la congestión hay capacidad para **una** de las dos transmisiones, y hay que **interrumpir la del 9002** sin que A ni B se enteren, aceptando pérdida de información. El enunciado sugiere alterar los **campos de secuencia** de los datos y de los ACKs.

La idea en una frase: el router **descarta** los datos del 9002 y **les miente a los dos extremos**: le **falsifica un ACK** a A diciendo que los recibió, y le **resta a B** los bytes descartados, de modo que **B no note el hueco**.

```
A manda 1000..1099 → R lo descarta (dif_SN = 100) y manda a A un ACK 1100 FALSO
A cree que recibió todo y sigue con 1100..1199 → R lo pasa a B con SN = 1100..1199 (1200 −
B responde ACK 1200 → R lo corrige hacia A como ACK 1100 → A lo retransmite → R lo descarta
```

Las dos invariantes: (1) **A nunca ve un hueco**, porque R le manda un ACK por cada byte que descarta; (2) **B nunca ve un hueco**, porque R le resta a los datos hacia B la cantidad total de bytes descartados (`dif_SN`) y suma esa misma cantidad a los ACKs hacia A, para que todo siga cuadrando con lo que A cree haber enviado.

Por qué se puede perder información en silencio: en TCP **el byte nunca se confirma uno por uno**; lo que importa es la **numeración**. Si R la mantiene coherente de los dos lados, ni A ni B detectan el hueco. Es el mismo principio que usa un proxy TProxy.

Dos errores de la solución oficial publicada, que son material de examen: el `select` tiene la condición **invertida** (`if ((status = select(sel)) <> -1) { error; exit(); }` mata el proceso cuando `select` **tiene éxito**), y el ACK falso se arma con el `largo` **ya restado por la cabecera** y con el `ACK` de un paquete de datos en el campo `SN`. A eso se suman que **no retransmite** el ACK falso ni **reinicia** `dif_SN`: los dos bugs que hay que mirar en cualquier implementación de este tipo.

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. TCP tiene **20 bytes** mínimos y compra con ellos orden, ACK, control de flujo, checksum y ciclo de vida de la conexión.
2. **seq** y **ack** son de **32 bits y cuentan BYTES**: el ACK es el **próximo byte esperado** y **acumulativo**.
3. Los **ACKs duplicados** evidencian un hueco y disparan el *fast retransmit*.
4. **Data Offset** va en **palabras de 32 bits** (`5` = 20 bytes); el **byte 12** tiene offset + reservados + NS y el **byte 13** los flags (CWR y ECE reservados; URG, ACK, PSH, RST, SYN, FIN).
5. **PSH** significa "no esperes a llenar el buffer", **no** "entregá esto ahora": segmentación, WAL del emisor y buffering del receptor rompen la frontera de mensaje.
6. **Opciones**: MSS (2), Window Scale (3), SACK permitted (4), Timestamps (8), con su RFC.
7. El **checksum** es un Internet checksum de complemento a uno **con pseudo-cabecera**, **no un CRC**: ve todos los errores de un bit, pero no todos los múltiples.
8. Si el checksum falla, el segmento **se descarta en silencio** y para el emisor es indistinguible de uno perdido.
9. `RFC 9293` **(2022)** sustituye a la `RFC 793`; `RFC 7323` obsolesce a la 1323; SACK es `RFC 2018`.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. Un segmento con `seq = 1100` y 20 bytes de datos. ¿Qué ACK devuelve el receptor?

<details>
<summary>Respuesta</summary>
</details>

2. Si `Data Offset` vale 5, ¿cuántos bytes de cabecera tiene el segmento?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Qué le dice realmente el flag PSH?

<details>
<summary>Respuesta</summary>
</details>

4. ¿Por qué se pone el MSS en el SYN y no en todos los segmentos?

<details>
<summary>Respuesta</summary>
</details>

5. Calculá el checksum de un segmento cuya suma (sin el campo checksum) da `0x7FF0`.

<details>
<summary>Respuesta</summary>
</details>

6. Un segmento llega con el checksum corrupto. ¿Qué hace el receptor y qué hace el emisor?

▶ Respuesta

7. ¿Por qué el checksum de TCP es un Internet checksum y no un CRC como el de Ethernet?

▶ Respuesta

8. ¿Por qué no basta con tres de los cuatro valores de la 4-tupla para elegir un socket TCP?

▶ Respuesta

🔗 Referencias

- **Examen** `eirc1102` / `sirc1102` — P3 (10 pts, campos más importantes de TCP y UDP)
- **Examen** `eirc0703` / `sirc0703` — P4 (10 pts, lo mismo, con la lista de 13 + 3 campos)
- **Examen** `eirc0112` / `sirc0112` — Problema 2 (PUSH) · Problema 1 (router que interrumpe una conexión TCP)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6 — §6.3 (*Segmentos TCP*), §6.5
- `RFC 9293` — *TCP* (2022, *Internet Standard*; sustituye a la RFC 793) · `RFC 1122` — requisitos de los hosts · `RFC 1071` — Internet checksum
- `RFC 2018` — SACK (1996, *Standards Track*) · `RFC 7323` — Timestamps y Window Scale (2014; obsolesce la RFC 1323)
- `RFC 3168` / `RFC 9768` / `RFC 9331` — ECN, AccECN y L4S · `RFC 9000` / `RFC 9114` — QUIC y HTTP/3: el **stream** reemplaza a la frontera de mensaje de TCP
- **Clase siguiente:** 18 — TCP: establecer y cerrar la conexión